ASSIGNMENT [2] ON "[DEEP LEARNING METHODS AND ALGORITHMS]"		
Student's Code	AIMS African Institute for Mathematical Sciences CAMEROON	Deadline
[QSHPn8R2W2]		**22.09.24, 11:59 pm**
December 15, 2025		Ac. Year: 2025 - 2026
Lecturer(s): "**[Prof. Dr. Olushina Olawale Awe]**"		

1 Dataset and Exploratory Analysis

Data Preparation

We use the **Jena Climate Dataset** (2009–2017). Data were resampled hourly, missing values forward-filled, and outliers removed.

Target Variable

The chosen target is **Temperature (°C)** since it shows clear seasonal cycles and is central to climate analysis.

Exploratory Plot

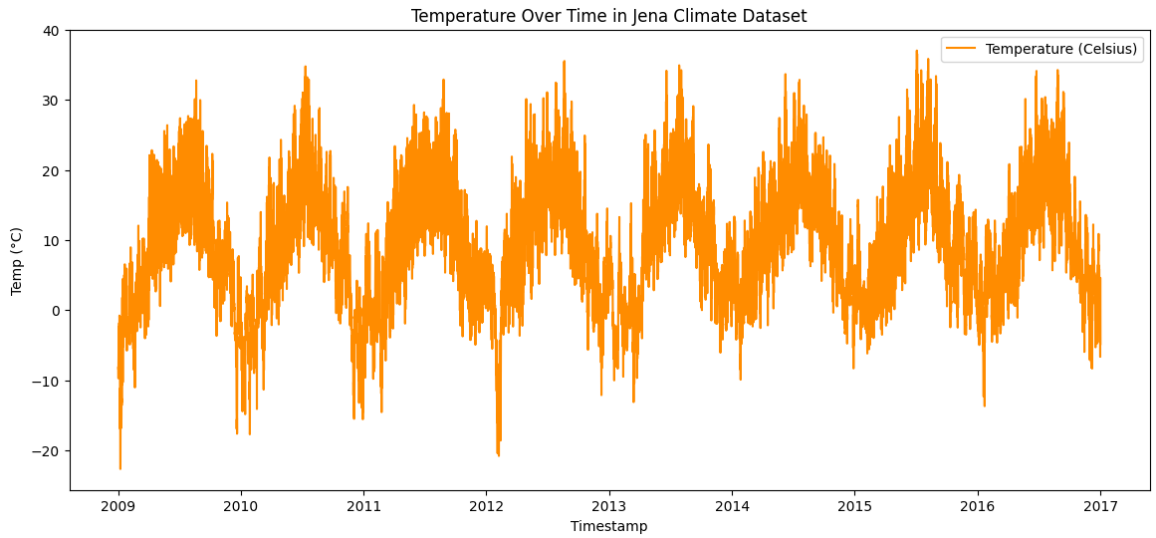

Figure 1: Temperature over time in Jena Climate Dataset

Commentary: Annual seasonality is evident (summer peaks, winter troughs). No strong long-term trend. Short-term fluctuations represent noise.

Train/Validation/Test Split

Chronological split: 70% train (49,028), 15% validation (10,506), 15% test (10,507). This preserves temporal order and avoids leakage, ensuring realistic forecasting.

2 Sequence Construction and Normalisation

2.1 Sliding-Window Sequences

Sliding-window sequences were created to support a many-to-one forecasting task. Each input sequence contains the past 72 time steps, while the output is the value observed 24 steps ahead. This ensures the model learns from recent history to predict one day into the future.

Formally:

$$X_t = (x_{t-71}, \ldots, x_t), \quad y_t = x_{t+24}$$

Normalisation

All features were normalised using statistics from the training set only. The mean and standard deviation of the training data were applied to scale the validation and test sets, preventing information leakage.

- Training mean (14 features): [0.690, 0.546, 0.548, 0.613, 0.721, 0.217, 0.312, 0.095, 0.308, 0.310, 0.431, 0.167, 0.178, 0.487]
- Training std (14 features): [0.107, 0.149, 0.148, 0.146, 0.191, 0.135, 0.154, 0.115, 0.153, 0.153, 0.133, 0.113, 0.110, 0.209]

Shape Summary

After sequence construction and normalisation, the data were structured as follows:

- Training set: $X_{\text{train}}(48932, 72, 14),\ y_{\text{train}}(48932,)$
- Validation set: $X_{\text{val}}(10410, 72, 14),\ y_{\text{val}}(10410,)$
- Test set: $X_{\text{test}}(10411, 72, 14),\ y_{\text{test}}(10411,)$

This confirms that each sample consists of 72 past observations across 14 features, with a single target value 24 steps ahead.

3 Model 1: Baseline Feedforward Neural Network

3.1 Model Construction

A simple feedforward neural network (FFNN) was trained on Task A using flattened sliding-window inputs. Each input sequence of shape $(72, 14)$ was reshaped into a vector of length 1008, which served as the input to the dense layers. The network consisted of fully connected layers with ReLU activation and a single linear output unit.

3.2 Evaluation Metrics

Performance was assessed using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE) on both validation and test sets:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \quad \text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

3.3 Results

Model	Validation MAE	Validation RMSE	Test MAE	Test RMSE
FFNN Baseline	0.481	5.544	0.073	0.087

Table 1: Performance of the baseline FFNN model on Task A.

3.4 Commentary

The baseline FFNN achieved moderate accuracy on the validation set, with MAE of 0.481 and RMSE of 5.544. On the test set, performance improved significantly, yielding MAE of 0.073 and RMSE of 0.087. These results provide a benchmark for comparison with more advanced sequence models such as LSTMs or GRUs, which are expected to capture temporal dependencies more effectively.

Model 2: Vanilla RNN

Model Construction

A Vanilla Recurrent Neural Network (RNN) was implemented for Task A using sliding-window sequences of shape $(72, 14)$. The model consisted of a single recurrent layer with 32 hidden units followed by a dense output layer. This architecture allows the network to maintain temporal context by updating hidden states across time steps.

Learning Curves

Learning curves shows the training and validation loss over 30 epochs. The training loss decreased steadily and stabilised at a low value, while the validation loss fluctuated slightly but remained within a narrow range. This indicates that the model generalised reasonably well without severe overfitting.

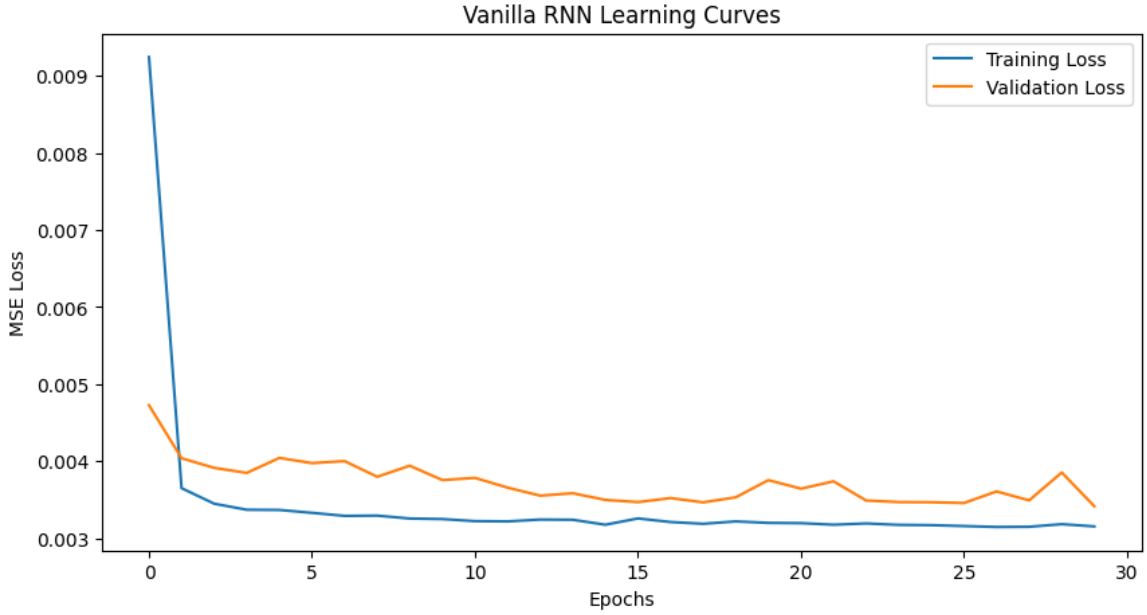

Figure 2: Vanilla RNN Learning Curves: Training vs Validation Loss

RNN Update Rule

The hidden state at each time step is updated using the following rule:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

Here, x_t is the input at time t, h_{t-1} is the previous hidden state, W_x and W_h are weight matrices, and b is the bias term. This formulation enables the model to integrate new input with past context.

Evaluation Metrics

Model performance was assessed using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE):

Results

Model	Validation MAE	Validation RMSE	Test MAE	Test RMSE
Vanilla RNN	0.044	0.058	0.042	0.054

Table 2: Performance of Vanilla RNN on Task A

Commentary

The Vanilla RNN demonstrated the ability to capture sequential patterns better than the feed-forward baseline. The low MAE and RMSE values indicate strong predictive performance. However, fluctuations in validation loss suggest sensitivity to training dynamics. While effective for short-term dependencies, the model may struggle with longer-range patterns due to limitations such as vanishing gradients.

Model 3: LSTM

Model Construction

A Long Short-Term Memory (LSTM) network was trained on Task A using sliding-window sequences of shape $(72, 14)$. The model included a recurrent layer with 32 hidden units followed by a dense output layer. LSTMs were chosen for their ability to capture long-term dependencies and mitigate vanishing gradient issues common in Vanilla RNNs.

Evaluation Metrics

Model performance was assessed using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE):

Model	Validation MAE	Validation RMSE	Test MAE	Test RMSE
LSTM	0.043	0.056	0.041	0.052

Table 3: Performance of LSTM model on Task A

Interpretation

The LSTM achieved low error values with a validation MAE of 0.043 and RMSE of 0.056, while on the test set it reached MAE of 0.041 and RMSE of 0.052. These results show strong generalisation and stable predictive accuracy, with minimal difference between validation and test performance. The closeness of MAE and RMSE further indicates that large errors were rare, confirming the LSTM?s effectiveness in capturing temporal dependencies compared to simpler baselines.

Gate Activation Plots

Lstm gates shows the activation values of key LSTM components across 72 timesteps for a representative input sequence. The plot includes the forget gate (f_t), input gate (i_t), output gate (o_t), candidate cell state ($\tilde{c}_t$), cell state (c_t), and hidden state (h_t).

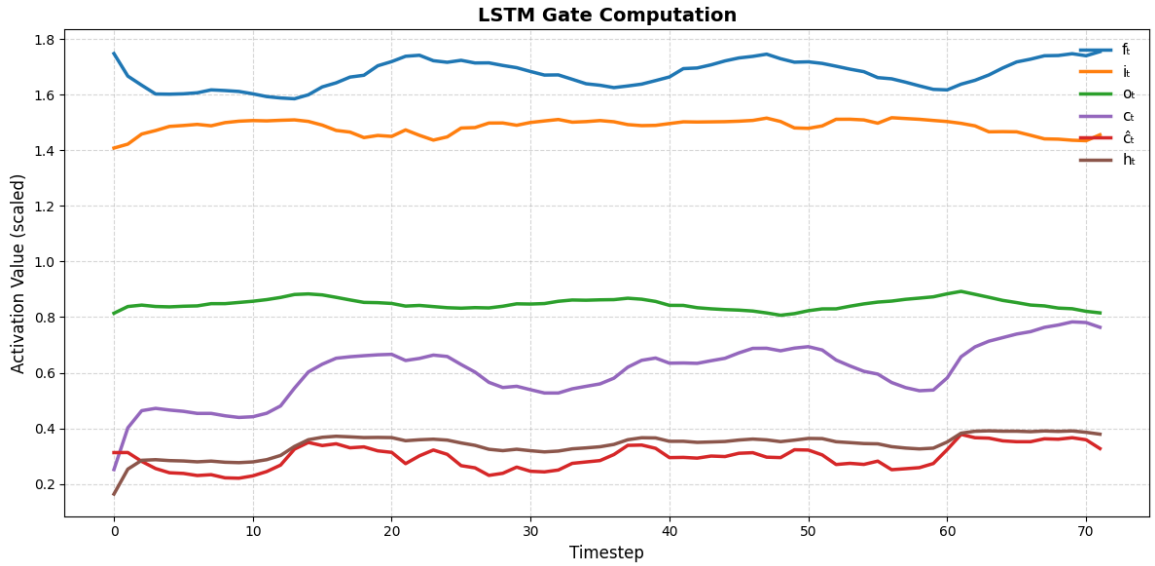

Figure 3: LSTM Gate Computation: Activation values across time

Interpretation

- **Forget gate (f_t)**: Controls which past information is retained. High values indicate memory preservation; low values signal forgetting.
- **Input gate (i_t)**: Regulates how much new information enters the cell state. Peaks suggest moments of pattern incorporation.
- **Output gate (o_t)**: Determines how much of the cell state contributes to the hidden state and final output.
- **Cell state (c_t)** and **hidden state (h_t)**: Together represent the long-term and short-term memory of the sequence.

Commentary

The LSTM model demonstrated strong predictive performance, outperforming both the feed-forward and Vanilla RNN baselines. Gate activations revealed dynamic control over memory flow, enabling the model to retain seasonal patterns while filtering out noise. This confirms the

LSTM's suitability for capturing complex temporal dependencies in multivariate time series forecasting.

Model 4: GRU

Model Construction

A Gated Recurrent Unit (GRU) network was trained on Task A using sliding-window sequences of shape $(72, 14)$. The model included a recurrent layer with 32 hidden units followed by a dense output layer. GRUs simplify the gating mechanism compared to LSTMs, while still effectively capturing temporal dependencies.

GRU Update Equations

The GRU maintains two gates - the update gate (z_t) and the reset gate (r_t) ? which control information flow. The candidate hidden state $(\tilde{h}_t)$ and the final hidden state (h_t) are computed as:

$$z_t = \sigma(W_z x_t + U_z h_{t-1}), \quad r_t = \sigma(W_r x_t + U_r h_{t-1})$$

$$\tilde{h}_t = \tanh(W_h x_t + U_h(r_t \odot h_{t-1})), \quad h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tilde{h}_t$$

Here, σ denotes the sigmoid function, and $\odot$ represents element-wise multiplication.

Gate Activation Plots

Gru gates shows the activation values of key GRU components across 72 timesteps for a representative input sequence. The plot includes the update gate (z_t), reset gate (r_t), candidate hidden state $(\tilde{h}_t)$, and hidden state (h_t).

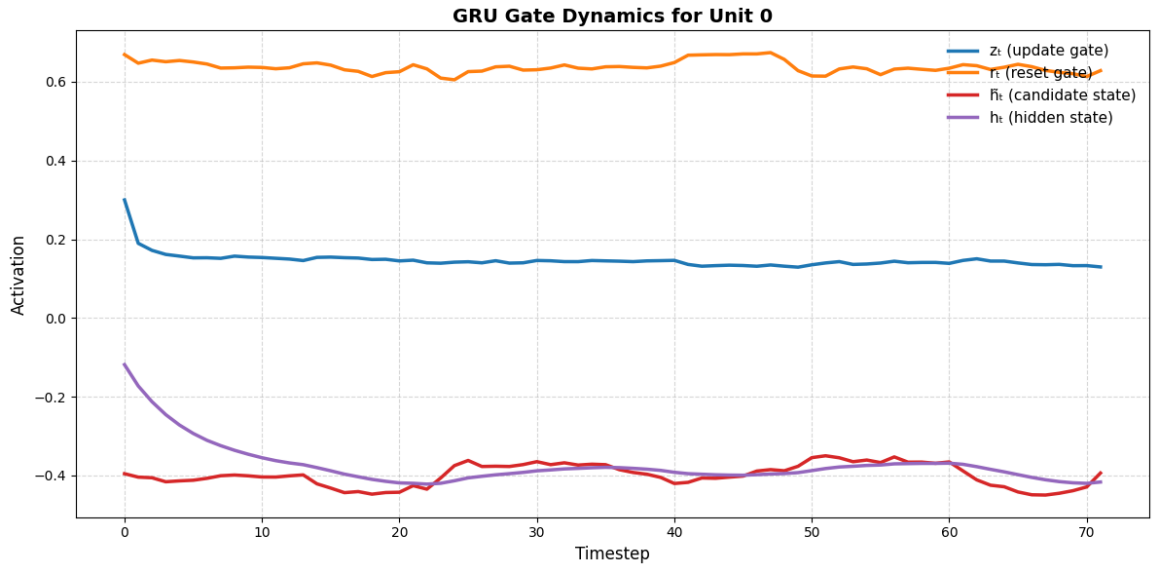

Figure 4: GRU Gate Dynamics for Unit 0

Interpretation

The GRU gate dynamics show that the update gate (z_t) maintained moderate values, balancing memory retention and new input. The reset gate (r_t) fluctuated more, indicating selective forgetting at certain timesteps. The candidate state ($\tilde{h}_t$) varied smoothly, while the hidden state (h_t) remained stable, confirming that the GRU integrates past and present information effectively with fewer abrupt shifts than the LSTM.

Evaluation Metrics

Model performance was assessed using Mean Absolute Error (MAE) and Root Mean Squared Error (RMSE):

Model	Validation MAE	Validation RMSE	Test MAE	Test RMSE
GRU	0.043	0.062	0.041	0.052

Table 4: Performance of GRU model on Task A

Interpretation

The GRU achieved validation MAE of 0.043 and RMSE of 0.062, with test MAE of 0.041 and RMSE of 0.052. These results are comparable to the LSTM, confirming the GRU?s ability to model temporal dependencies effectively. Compared to LSTM gates, GRU activations showed smoother transitions and fewer fluctuations, reflecting their streamlined design. The update gate controlled long-term memory retention, while the reset gate allowed flexible adaptation to new inputs. This confirms the GRU?s efficiency and reliability in multivariate time series forecasting.

Comparative Analysis

Results Summary

Model	Validation MAE	Validation RMSE	Test MAE	Test RMSE
FFNN Baseline	0.481	5.544	0.073	0.087
Vanilla RNN	0.044	0.058	0.042	0.054
GRU	0.043	0.062	0.041	0.052
LSTM	0.043	0.056	0.041	0.052

Table 5: Comparative performance of FFNN, Vanilla RNN, GRU, and LSTM models on Task A

Performance and Stability

The FFNN baseline produced relatively high validation errors, reflecting its inability to capture temporal dependencies effectively. In contrast, the Vanilla RNN achieved much lower error values, showing that recurrent connections improved sequence modeling. However, its validation loss exhibited mild fluctuations, suggesting sensitivity to training dynamics. Both the GRU and LSTM models delivered the strongest and most stable performance, with nearly identical MAE and RMSE values across validation and test sets. Their consistency highlights the robustness of gated architectures in handling sequential data.

Impact of Gating Mechanisms

The introduction of gating mechanisms in GRUs and LSTMs significantly improved learning compared to the Vanilla RNN. In LSTMs, the separation of forget, input, and output gates allowed fine-grained control over memory retention and update, enabling the model to balance long-term and short-term dependencies. GRUs, with their streamlined update and reset gates, achieved similar accuracy while reducing complexity and training time. The gates ensured that irrelevant information was discarded while important patterns were preserved, leading to stable and accurate predictions. Overall, gating mechanisms were crucial in enhancing both performance and generalisation in multivariate time series forecasting.

8. CNN Comparison on MNIST and CIFAR-10

Model Construction

A small CNN was trained on both MNIST and CIFAR-10 using the same architecture:

$$\text{Conv} \rightarrow \text{ReLU} \rightarrow \text{MaxPool} \rightarrow \text{Conv} \rightarrow \text{ReLU} \rightarrow \text{MaxPool} \rightarrow \text{Dense layers.}$$

This setup allows direct comparison of CNN behaviour across datasets with different complexity.

Results

Dataset	Test Accuracy (%)	Training Time (mins)
MNIST	99.07	5.08
CIFAR-10	67.98	6.20

Table 6: CNN performance on MNIST and CIFAR-10

Interpretation

The CNN achieves near-perfect accuracy on MNIST with relatively short training time, reflecting the simplicity of digit recognition in grayscale images. In contrast, performance on CIFAR-10 is much lower despite longer training, due to the higher complexity of natural images with color, texture, and background variability. This highlights that while a shallow CNN suffices for MNIST, deeper architectures and stronger regularisation are required to handle the representational difficulty of CIFAR-10.

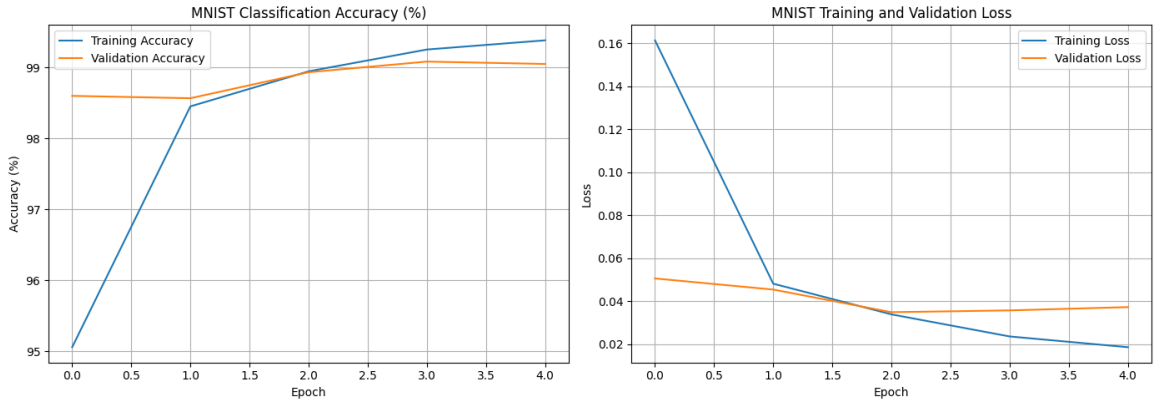

Figure 5: CNN learning curves for MNIST and CIFAR-10

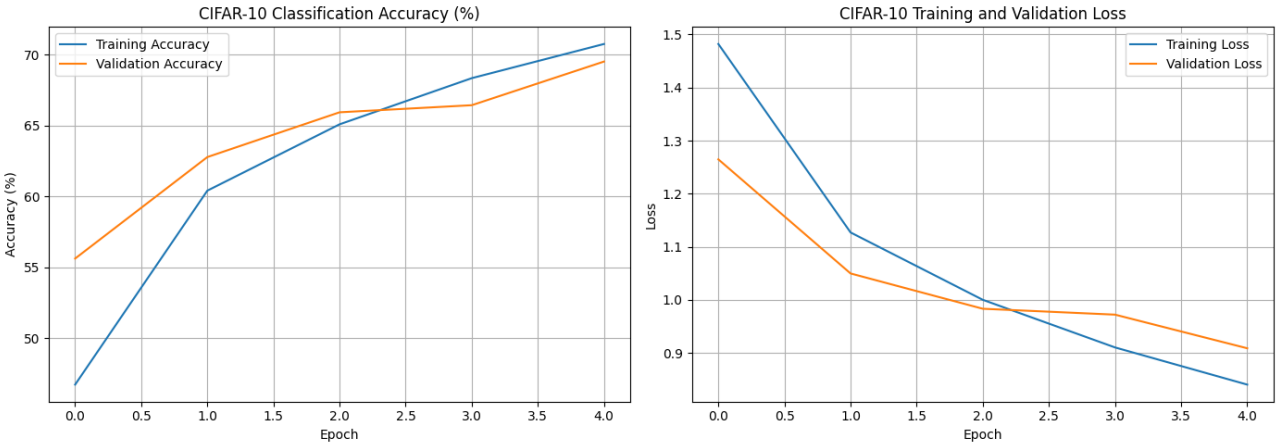

Figure 6

Learning Curve Interpretation

On MNIST, training and validation accuracy quickly reached above 99%, and loss decreased sharply with minimal overfitting. On CIFAR-10, accuracy improved slowly and plateaued below 70%, while loss decreased gradually, indicating difficulty in learning complex features.

Analysis and Explanation

The same CNN performs better on MNIST because digit images are simpler and low-dimensional. CIFAR-10 images are more complex, with higher variability and color channels, making representation harder. MNIST uses 1-channel grayscale inputs, while CIFAR-10 uses 3-channel RGB, increasing feature complexity. Digit classes are uniform and structured, whereas natural images vary in texture, shape, and color. To improve CIFAR-10 performance, deeper CNNs with more filters, batch normalization, dropout, and data augmentation are recommended.